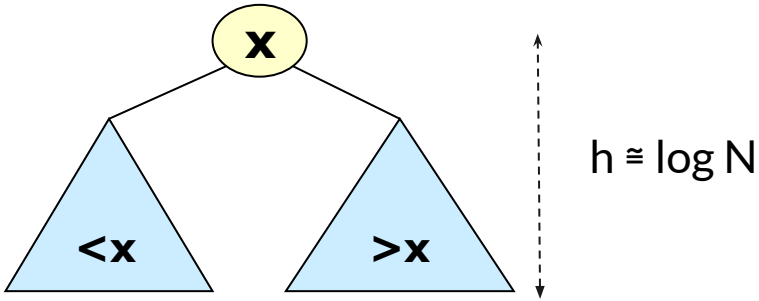
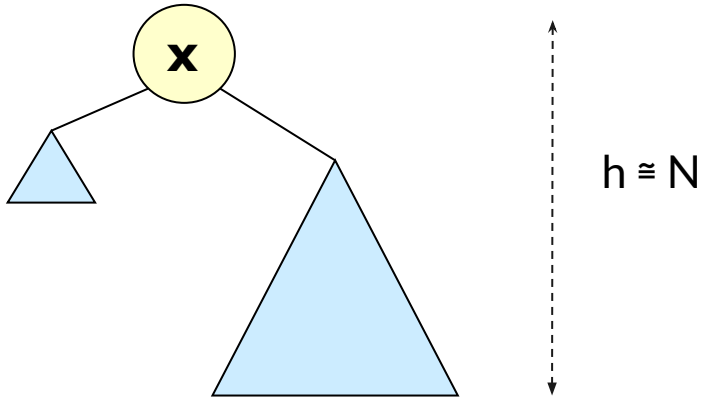
Árvores Balanceadas AVL

Estruturas de Dados

adalberto@computacao.ufcg.edu.br

Relembrando a busca em uma BST





Árvores Balanceadas AVL

Adelson-**V**elskii, G.; E. M. **L**andis:

An algorithm for the organization of information. Proceedings of the USSR Academy of Sciences 146: 263–266. (Russian) English translation by Myron J. Ricci in *Soviet Math. Doklady*, 3:1259–1263, 1962.

Adiciona um **invariante** a uma BST:

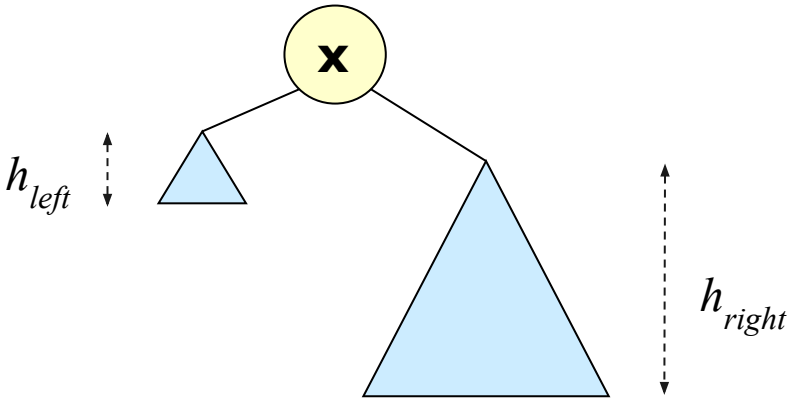
- Para todo nó a diferença de altura entre filho a esquerda (left) e filho a direita (right) é no máximo 1

$$|h(left) - h(right)| \leq 1$$

Árvores Balanceadas AVL

Fator de balanceamento

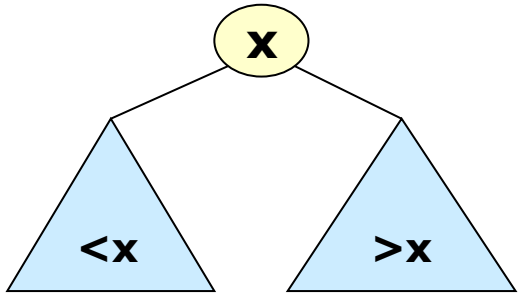
$$FB = h_{left} - h_{right}$$



Árvores Balanceadas AVL

Fator de balanceamento

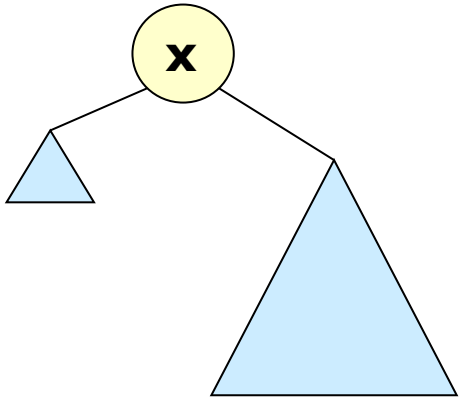
$$FB = h_{left} - h_{right} = 0$$



Árvores Balanceadas AVL

Fator de balanceamento

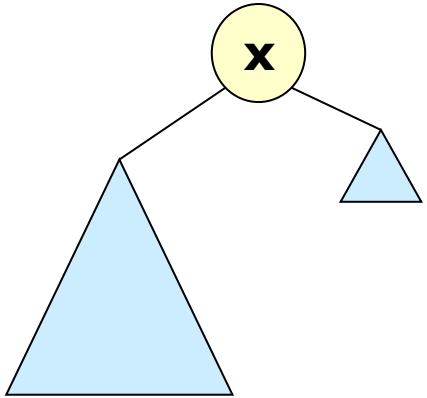
$$FB = h_{left} - h_{right} \leq 0$$



Árvores Balanceadas AVL

Fator de balanceamento

$$FB = h_{left} - h_{right} \geq 0$$



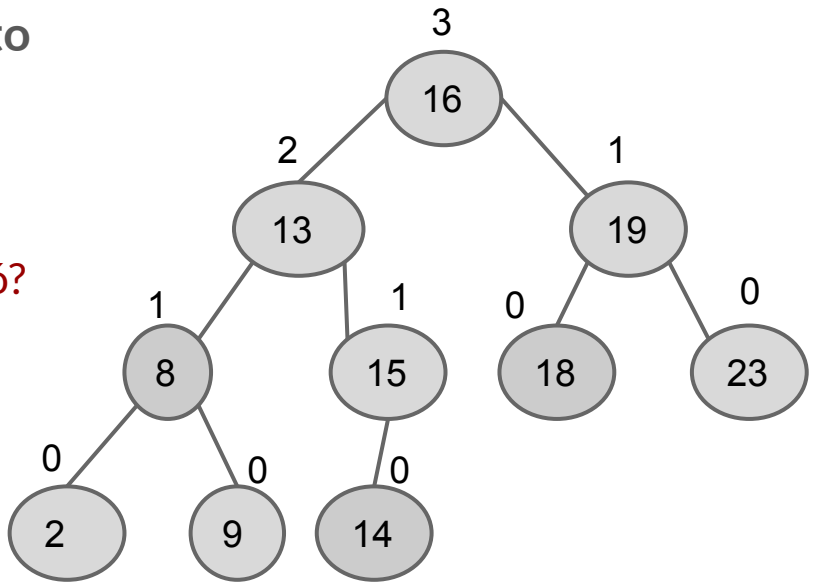
Árvores Balanceadas AVL

Fator de balanceamento

Altura de uma árvore vazia é -1.

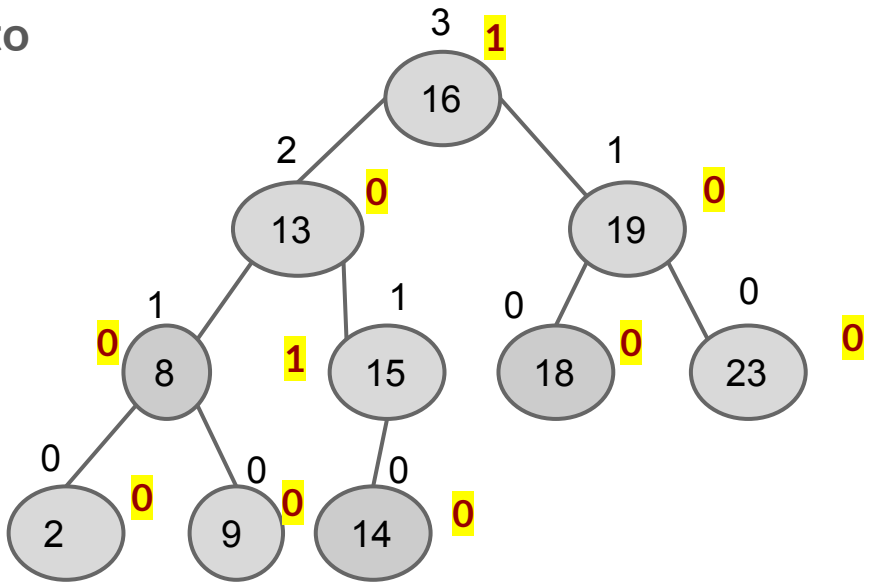
Como calcular a altura de um nó?

E a altura da árvore?



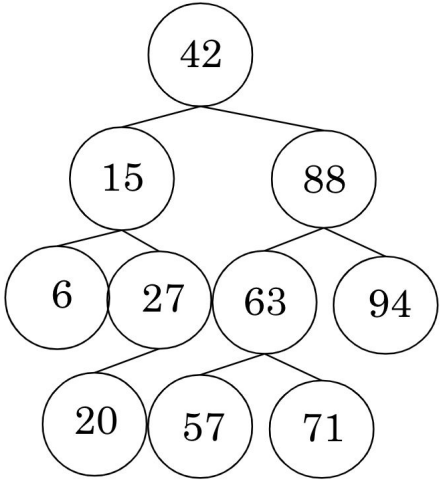
Árvores Balanceadas AVL

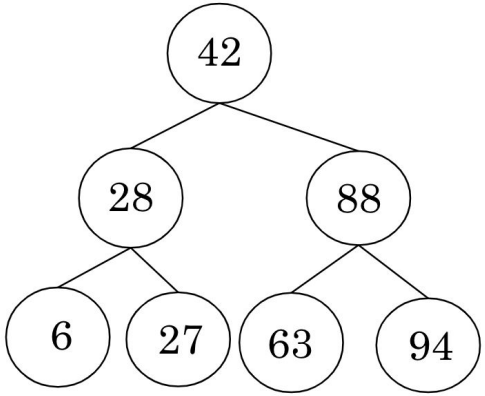
Fator de balanceamento

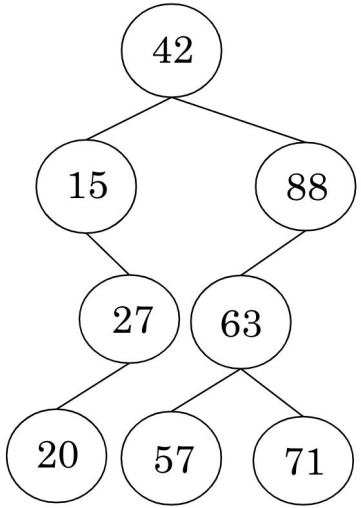


Árvores Balanceadas AVL

Quais são AVL?

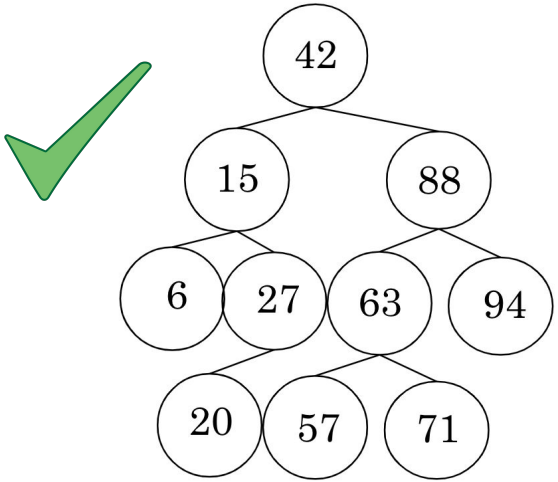


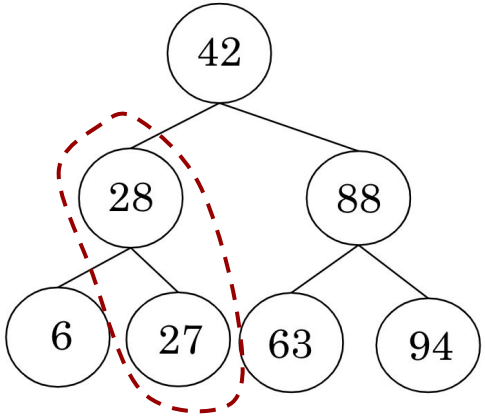


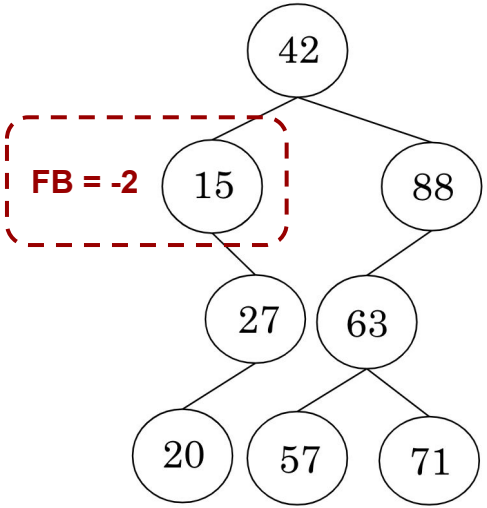


Árvores Balanceadas AVL

Quais são AVL?







Árvores Balanceadas AVL (Teorema)

$$\log N \leq h < 1.44.\log(N + 2) - 1.328$$

Simulação

Operações em Árvores AVL

Árvores AVL (operações)

Que operações podem violar o invariante?

$$|FB| \leq 1 \equiv |h_{left} - h_{right}| \leq 1$$

Como garantir o invariante da árvore!

Árvores AVL (operações)

$$|FB| \leq 1 \equiv |h_{left} - h_{right}| \leq 1$$

Que operações podem violar o invariante?

insert, remove

Como garantir o invariante da árvore!

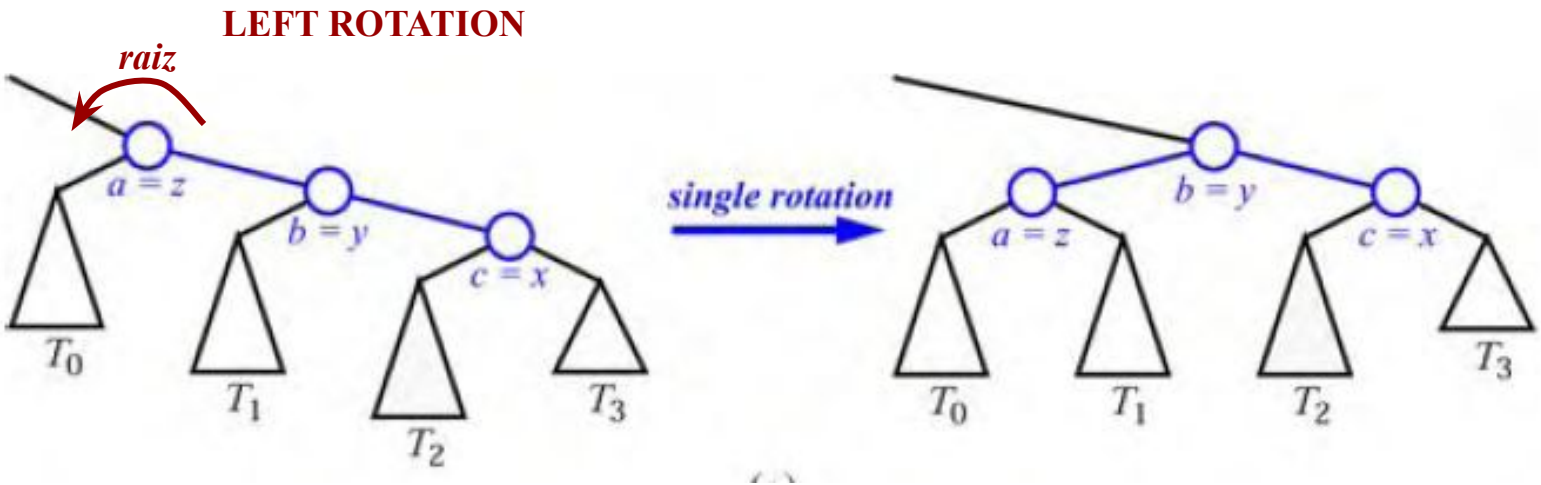
operações de rotação

Árvores AVL (Rotação)

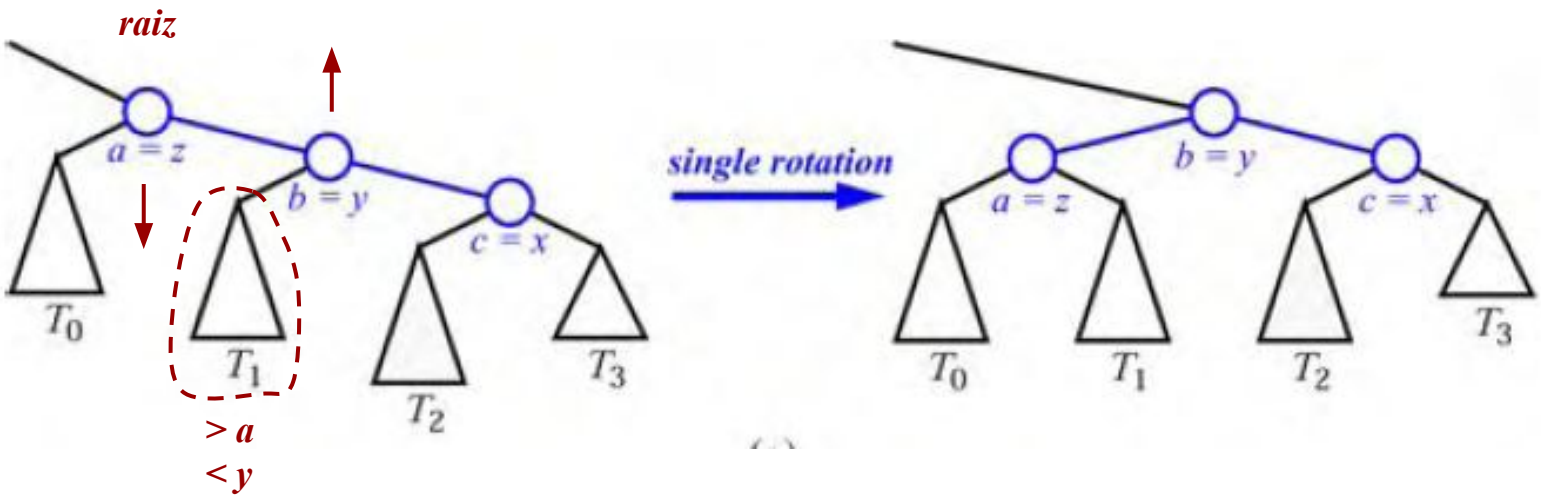
Operação focada em trocar um pai com algum filho

- Diminuir a altura de uma sub-árvore e aumentar a da outra
- Move um nó para baixo e outro para cima

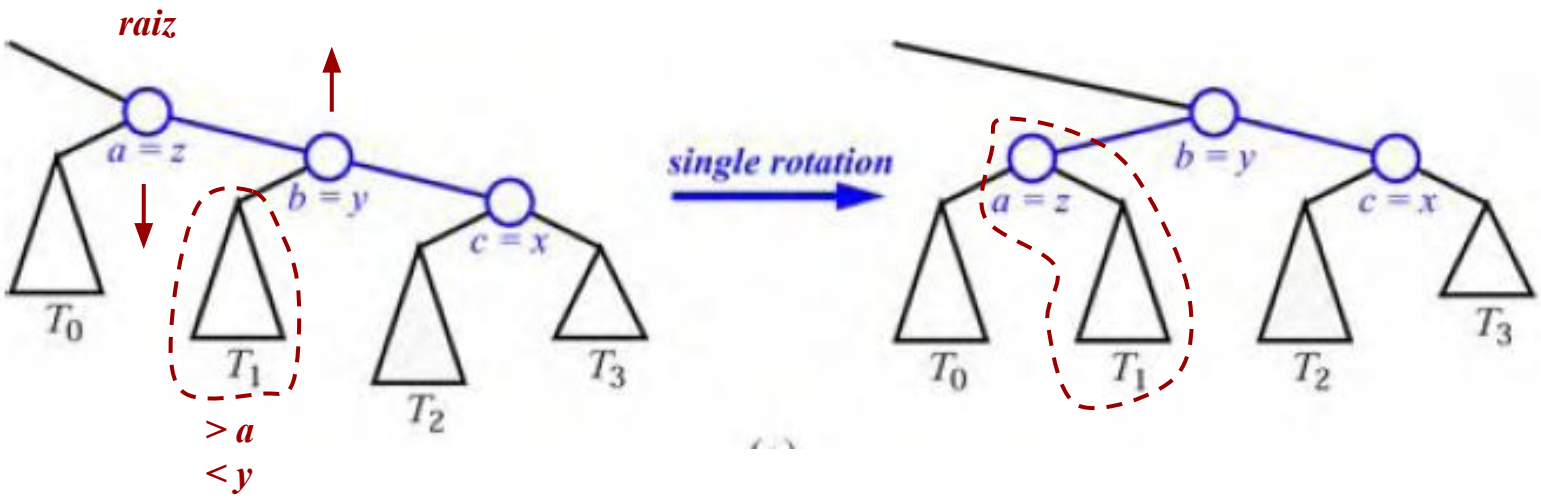
Árvores AVL (Rotação simples)



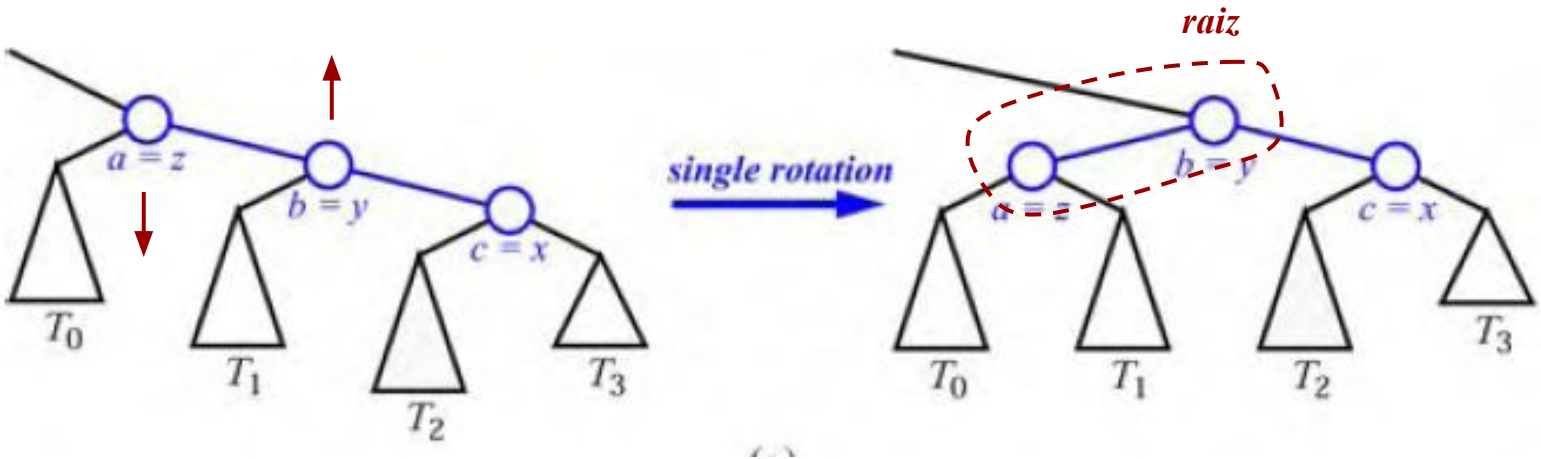
Árvores AVL (Rotação simples)



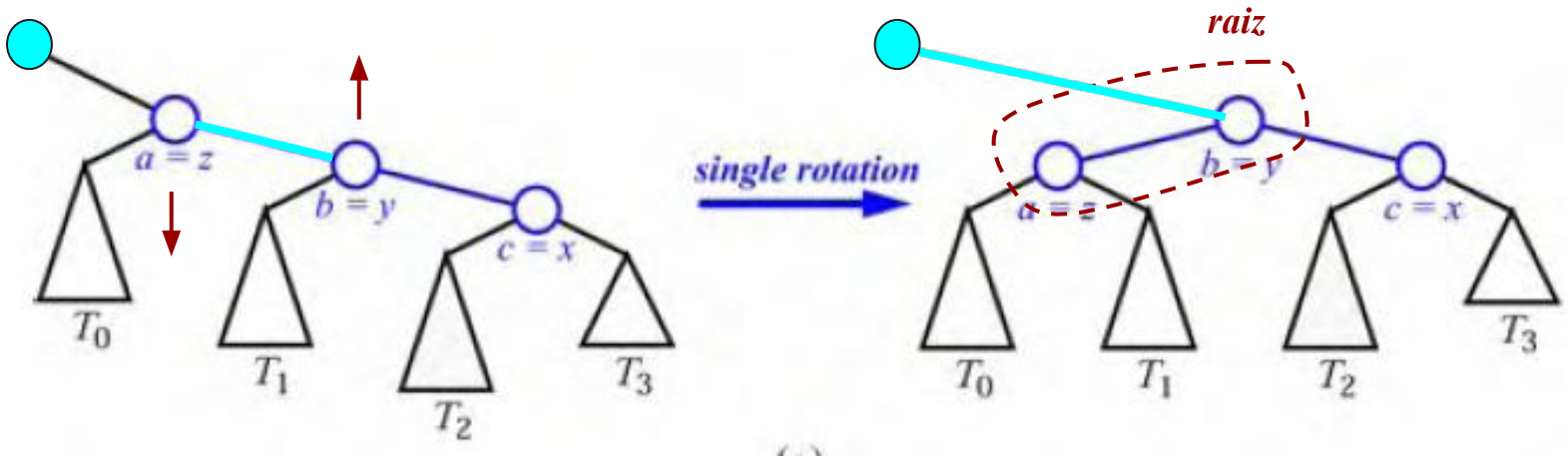
Árvores AVL (Rotação simples)



Árvores AVL (Rotação simples)

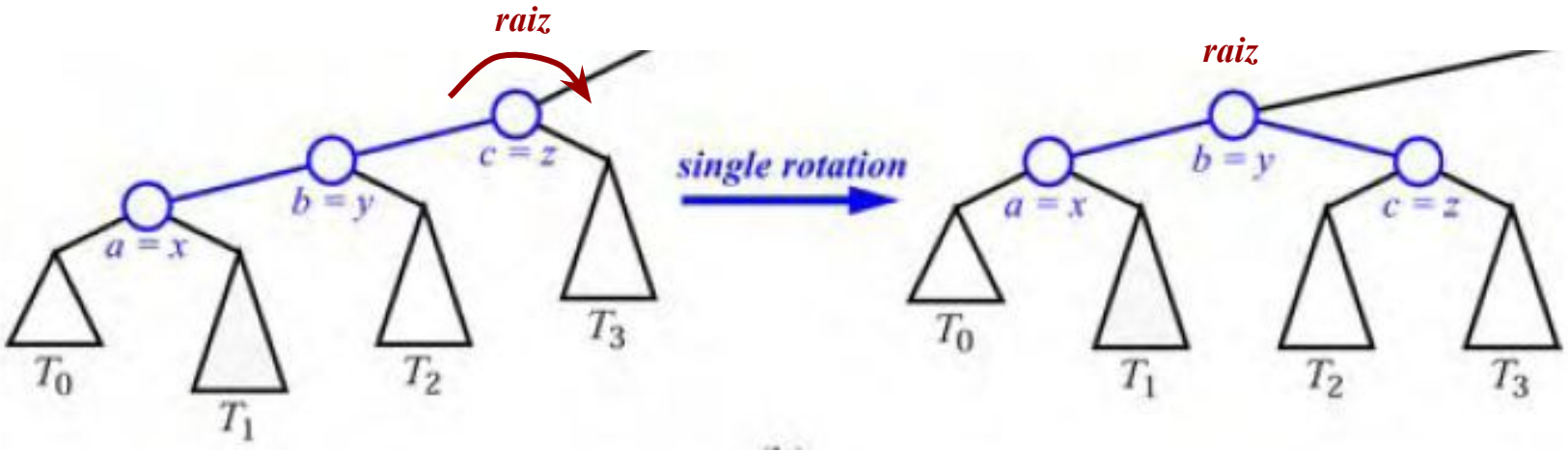


Árvores AVL (Rotação simples)



Árvores AVL (Rotação simples)

RIGHT ROTATION



Árvores AVL (Rotação Simples)

Generalização da rotação:

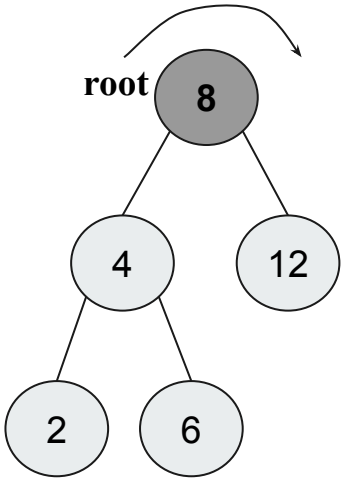
- Root – nó pai das sub-árvores a serem rotacionadas
- Pivot – o nó que será o novo pai
- RS – filho do mesmo lado da rotação (Rotation Side)
- OS – filho do lado oposto da rotação (Opposite Side)

Pseudo-código:

```
Pivot = Root.OS
Root.OS = Pivot.RS
Pivot.RS = Root
Root = Pivot
```

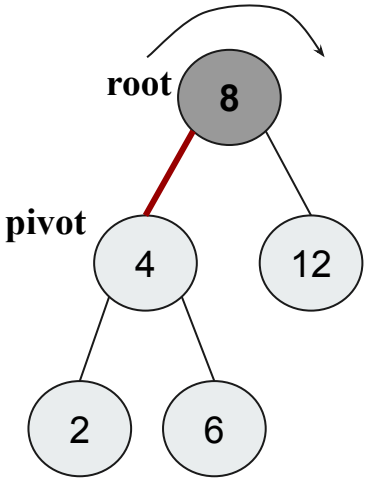
Árvores AVL (Rotação Simples)

```
Pivot = Root.OS
Root.OS = Pivot.RS
Pivot.RS = Root
Root = Pivot
```



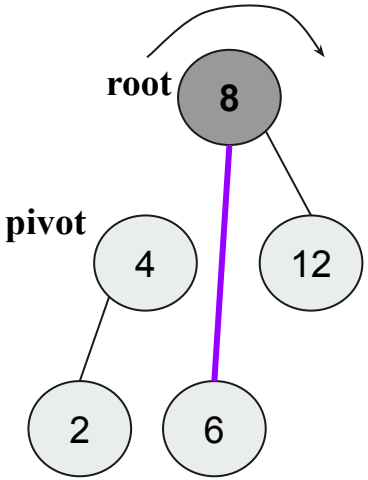
Árvores AVL (Rotação Simples)

```
Pivot = Root.OS
Root.OS = Pivot.RS
Pivot.RS = Root
Root = Pivot
```



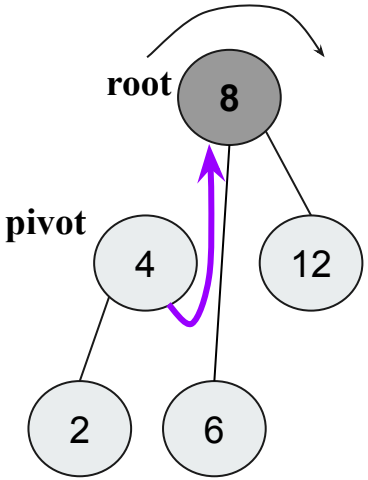
Árvores AVL (Rotação Simples)

```
Pivot = Root.OS
Root.OS = Pivot.RS
Pivot.RS = Root
Root = Pivot
```



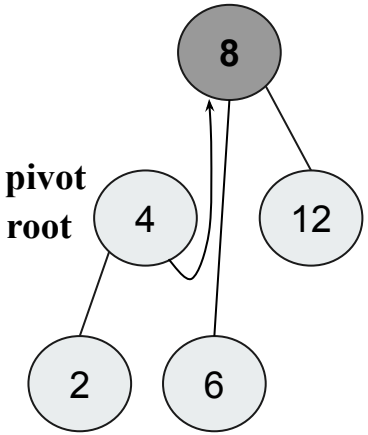
Árvores AVL (Rotação Simples)

```
Pivot = Root.OS
Root.OS = Pivot.RS
Pivot.RS = Root
Root = Pivot
```



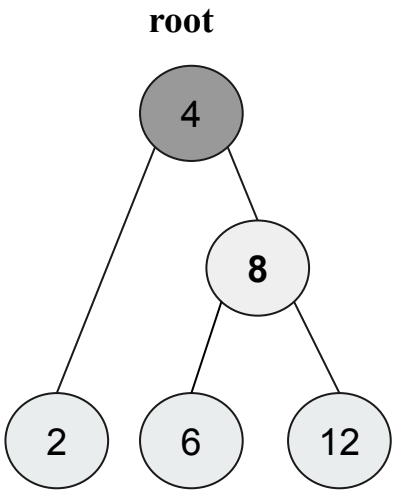
Árvores AVL (Rotação Simples)

```
Pivot = Root.OS
Root.OS = Pivot.RS
Pivot.RS = Root
Root = Pivot
```



Árvores AVL (Rotação Simples)

```
Pivot = Root.OS
Root.OS = Pivot.RS
Pivot.RS = Root
Root = Pivot
```



Complexidade = O(1)

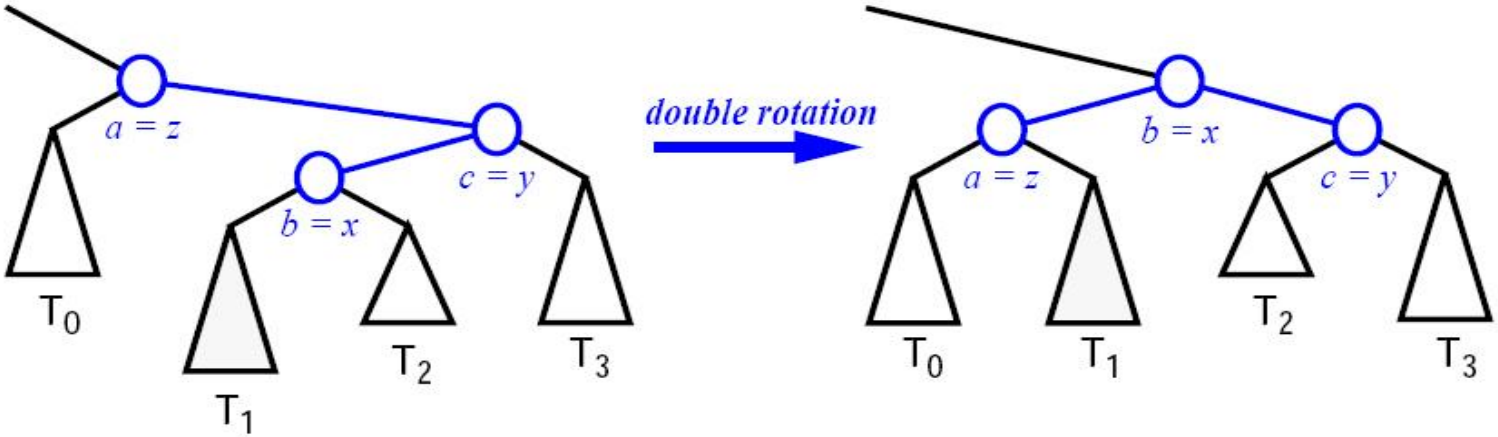
Tipos de Rotação Simples e Dupla

Árvores AVL (Rotações)

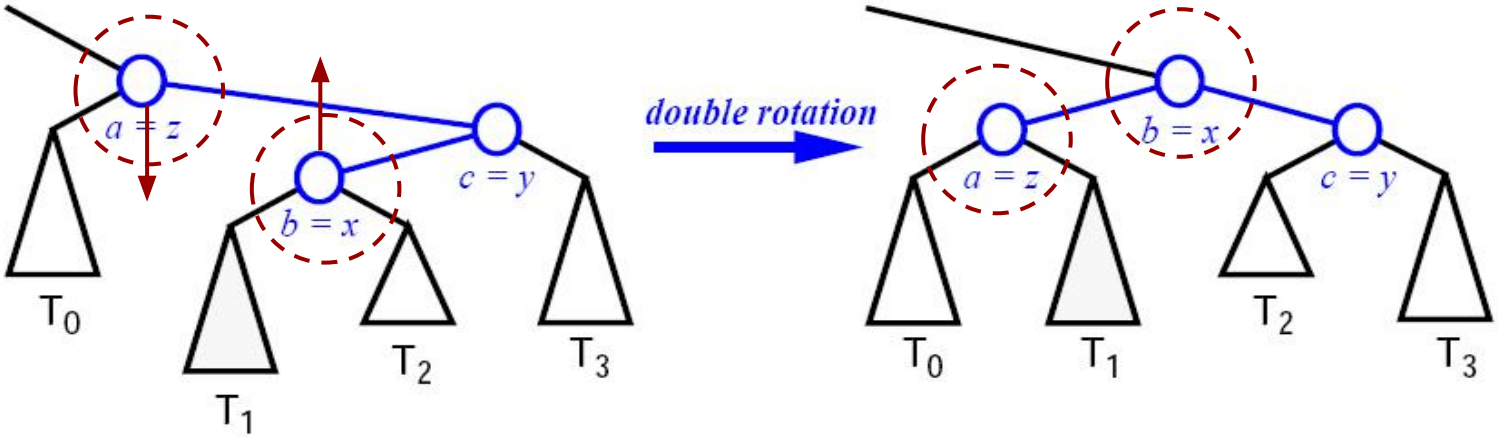
Tipos de rotações:

- Simples
  - Já visto (troca pai com filho)
- Dupla
  - Duas rotações simples (troca avô com neto)

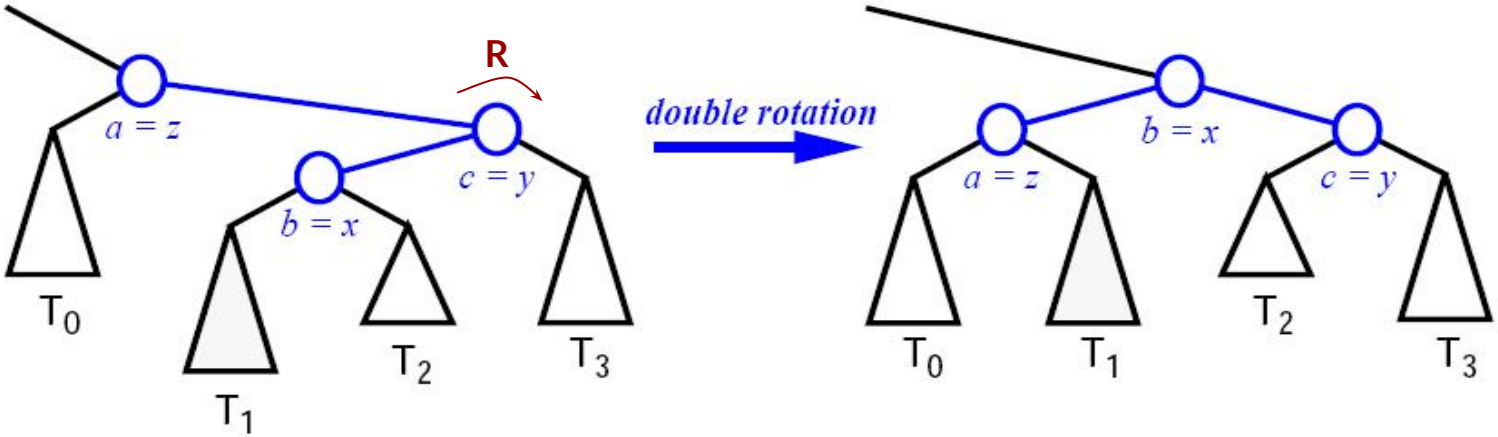
Árvores AVL (Rotações Duplas)



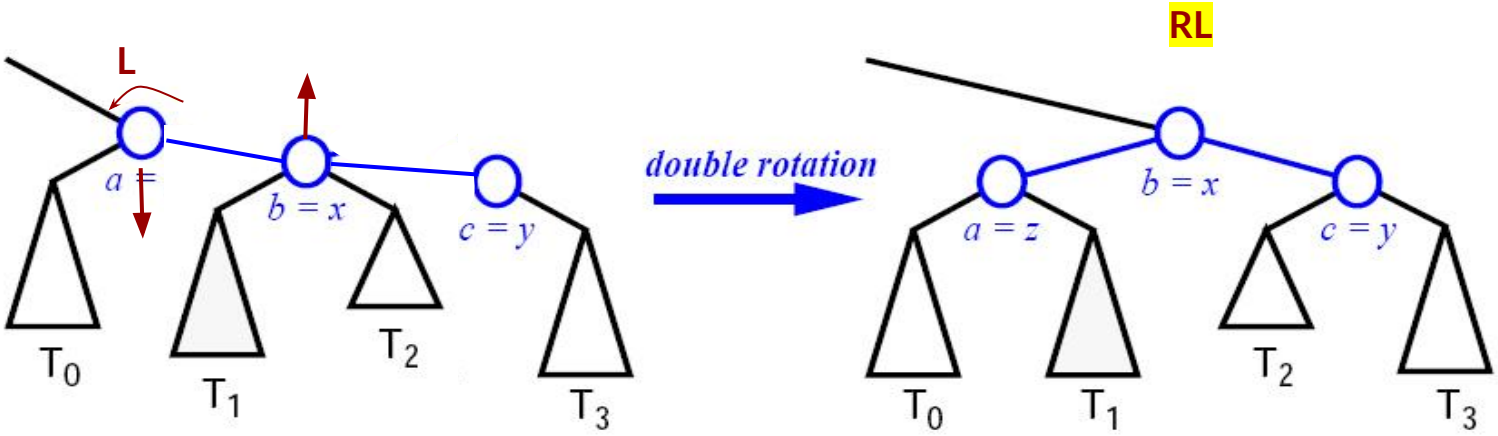
Árvores AVL (Rotações Duplas)



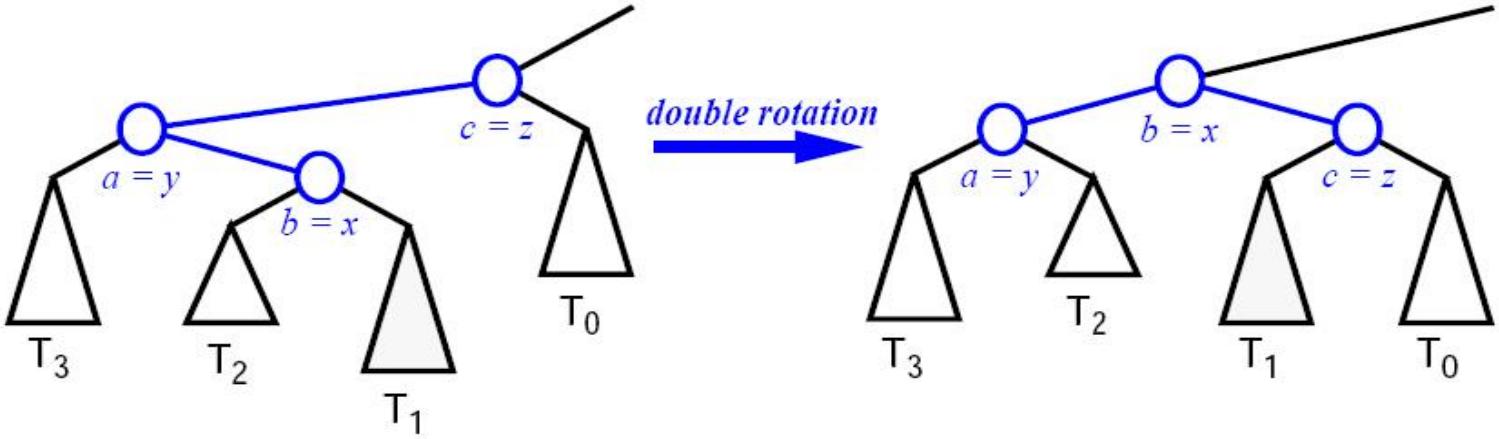
Árvores AVL (Rotações Duplas)



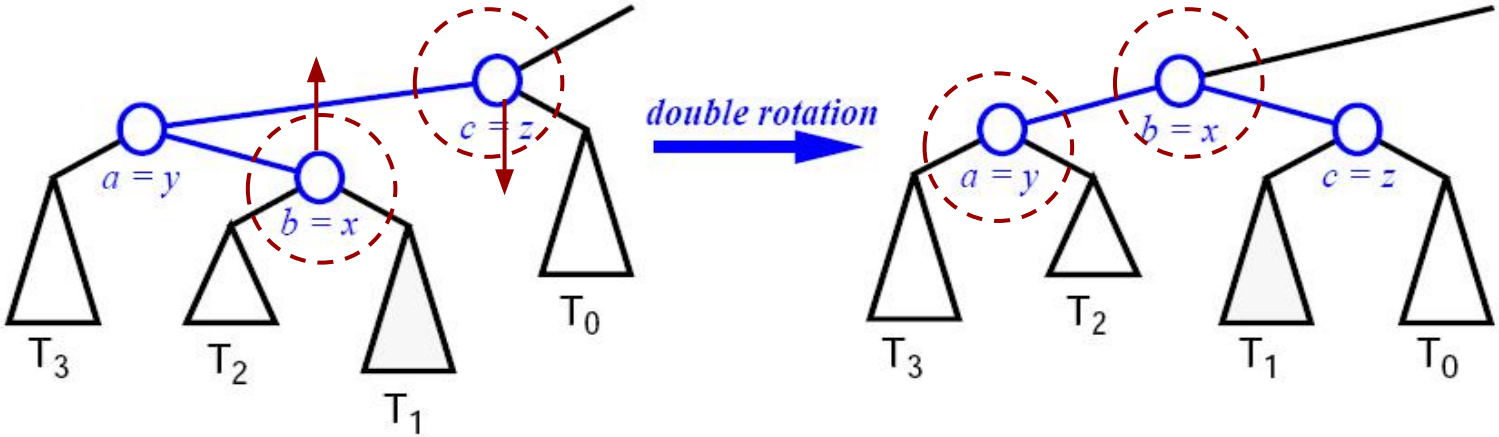
Árvores AVL (Rotações Duplas)



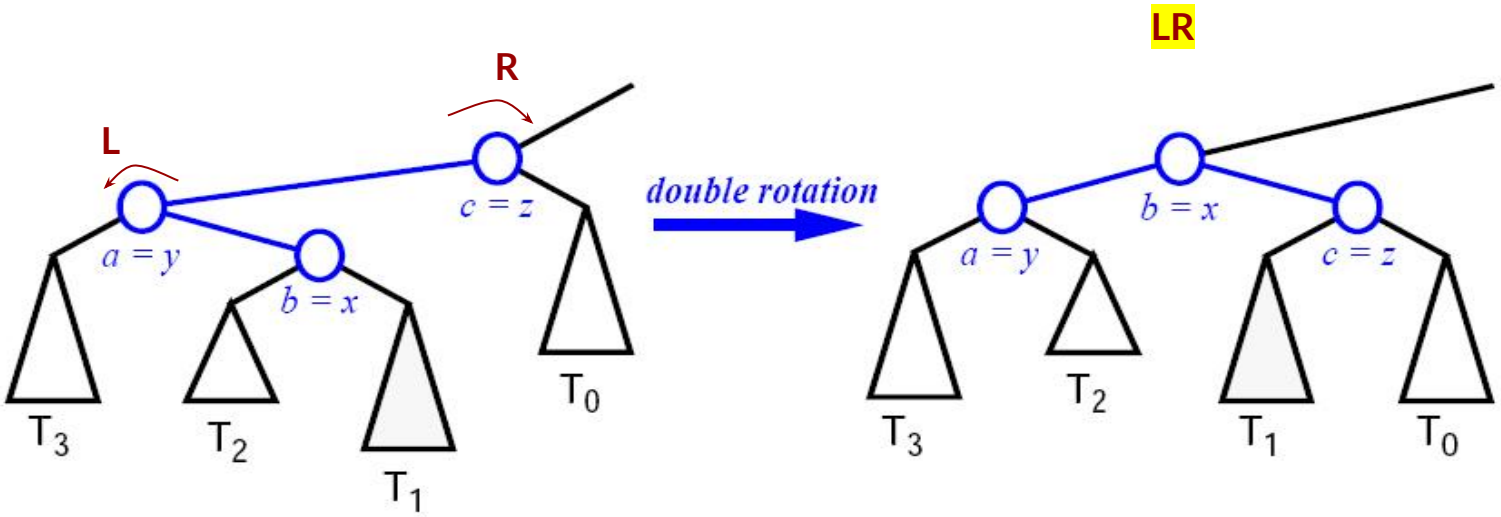
Árvores AVL (Rotações Duplas)



Árvores AVL (Rotações Duplas)



Árvores AVL (Rotações Duplas)

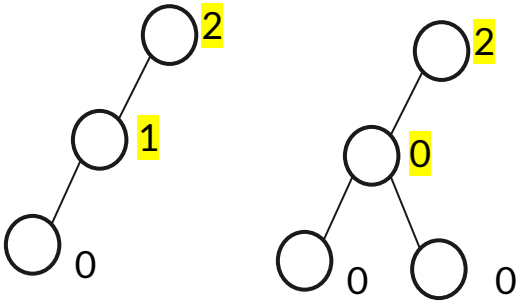


Árvores AVL (Resolver desbalanceamento)

Como consertar nós que estão desbalanceados?

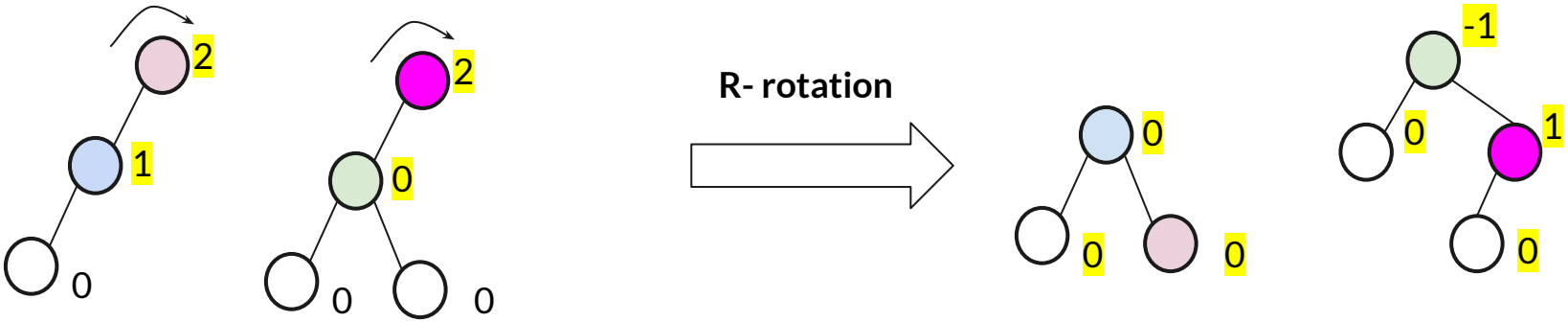
Árvores AVL (Resolver desbalanceamento)

LL - Left Left



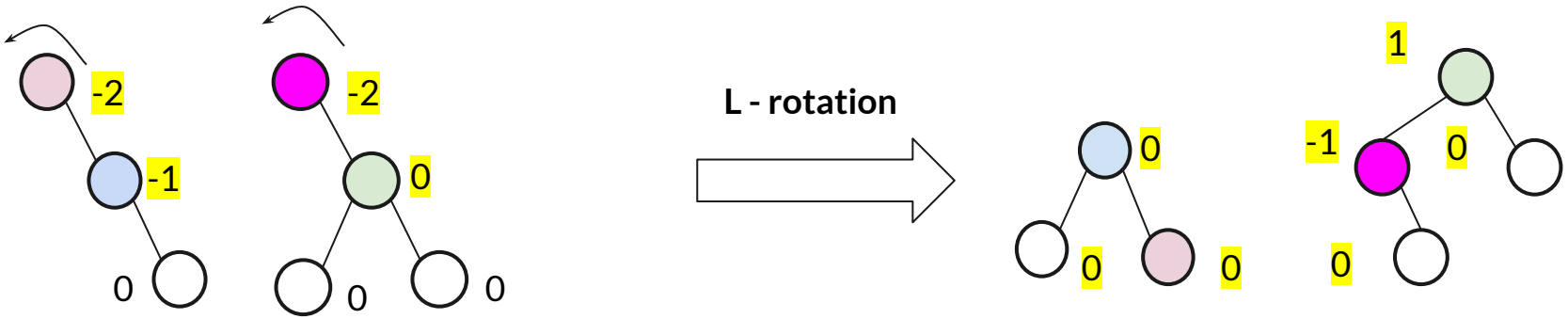
Árvores AVL (Resolver desbalanceamento)

LL - Left Left ⇒ **R rotation on root**



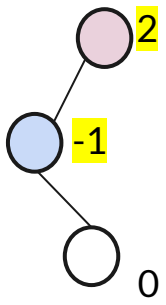
Árvores AVL (Resolver desbalanceamento)

RR - Right Right ⇒ **L rotation on root**



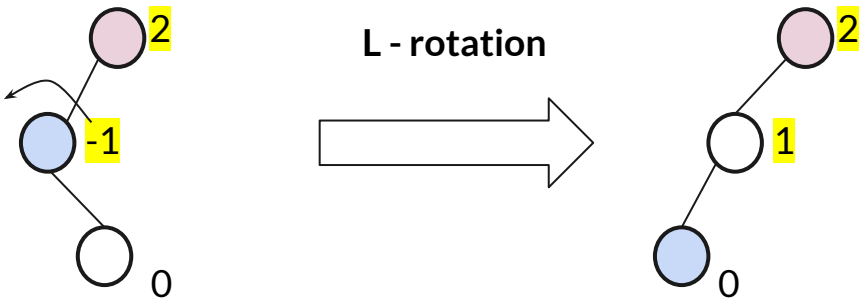
Árvores AVL (Resolver desbalanceamento)

LR - Left Right ⇒ **L(filho)R(root) rotation**



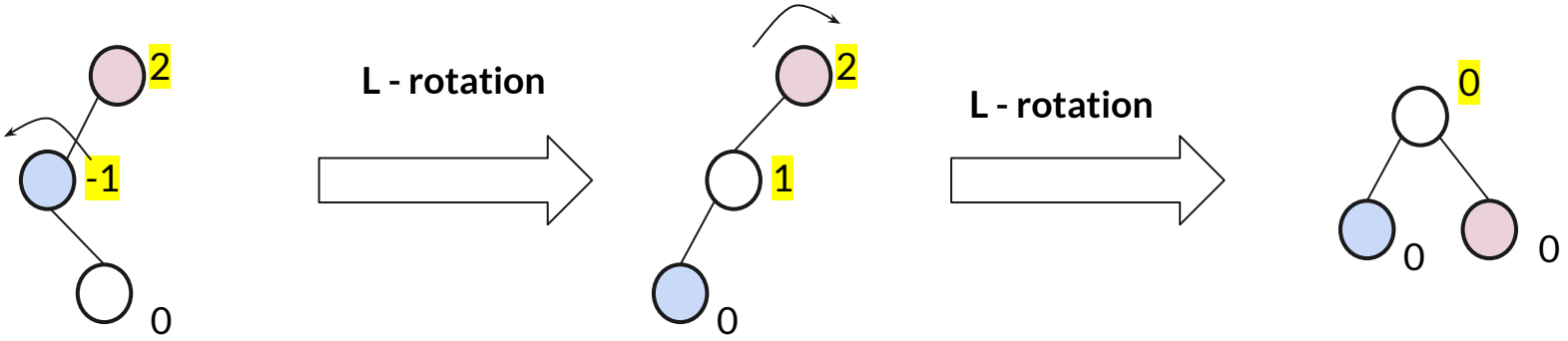
Árvores AVL (Resolver desbalanceamento)

LR - Left Right ⇒ **L(filho)R(root) rotation**



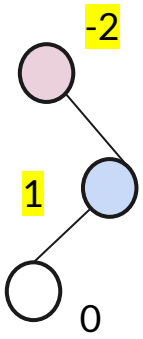
Árvores AVL (Resolver desbalanceamento)

LR - Left Right ⇒ **L(filho)R(root) rotation**



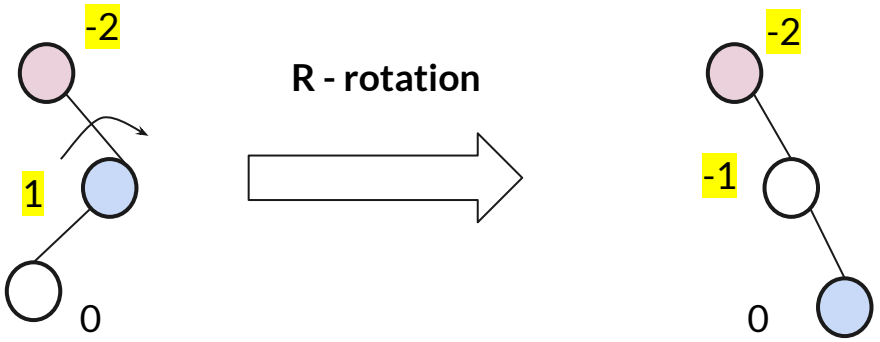
Árvores AVL (Resolver desbalanceamento)

RL - Right Left ⇒ **R(filho)L(root) rotation**



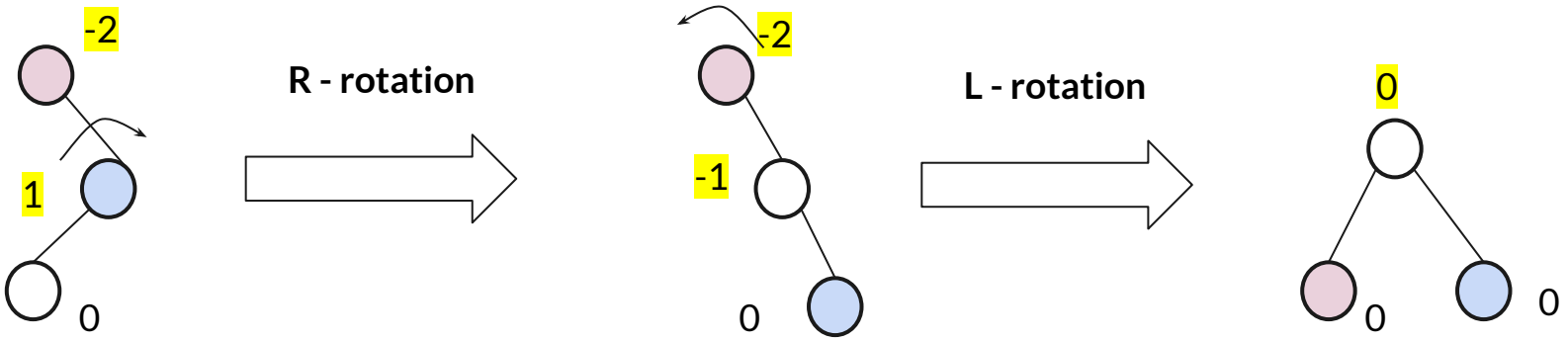
Árvores AVL (Resolver desbalanceamento)

RL - Right Left ⇒ **R(filho)L(root) rotation**



Árvores AVL (Resolver desbalanceamento)

RL - Right Left ⇒ **R(filho)L(root) rotation**



Árvores AVL (Resolver desbalanceamento)

Inserções acontecem como na BST (em uma folha)

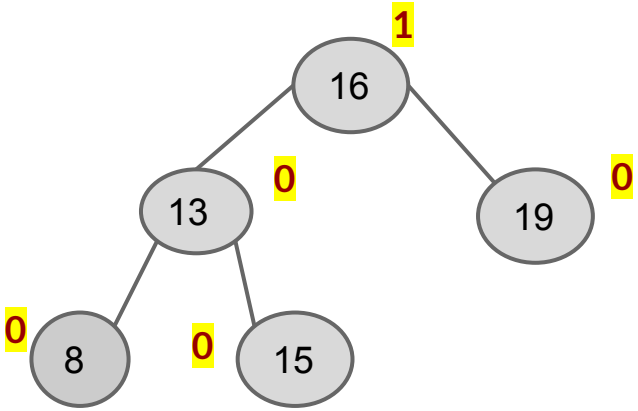
- Se desbalancear o nó (pai ou avô) → aplica rotação conveniente

Remoções acontecem como na BST

Se desbalancear o nó (pai ou avô) → aplica rotação conveniente

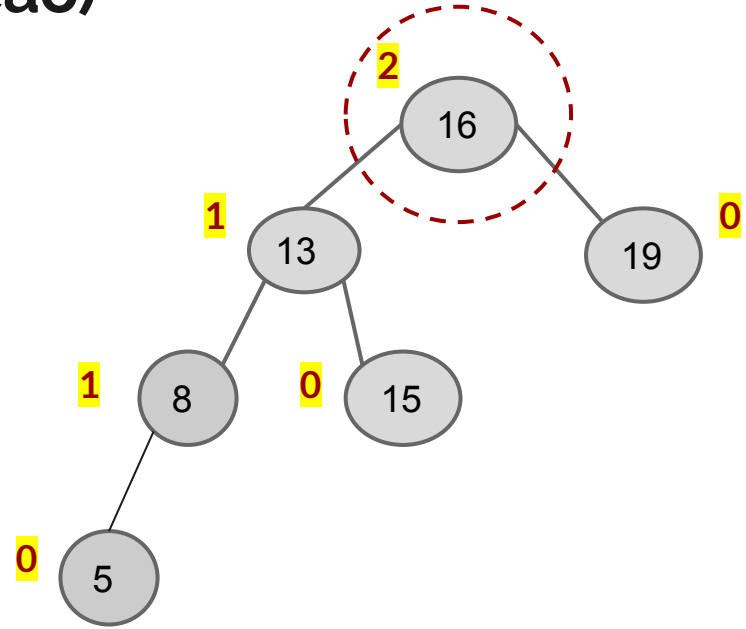
Árvores AVL (Inserção)

inserir(5)



Árvores AVL (Inserção)

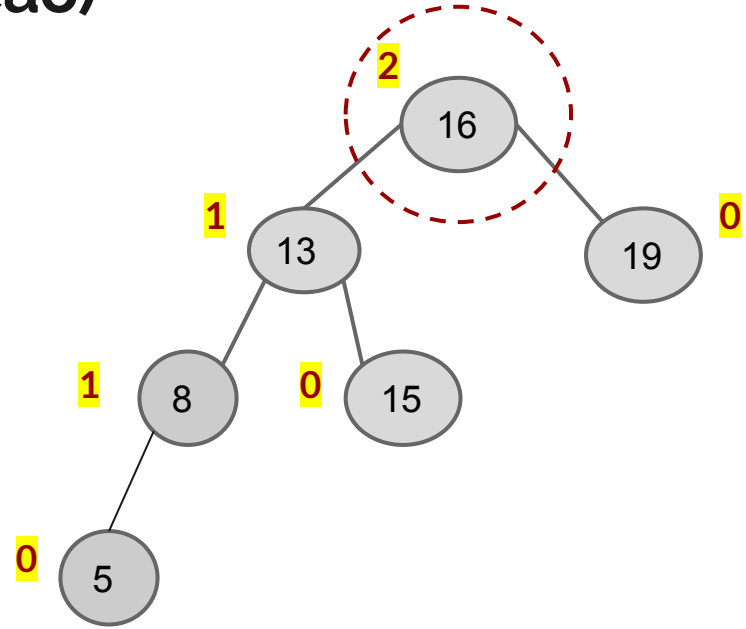
inserir(5)



Árvores AVL (Inserção)

inserir(5)

Caso LL ⇒ R-rotation (root)

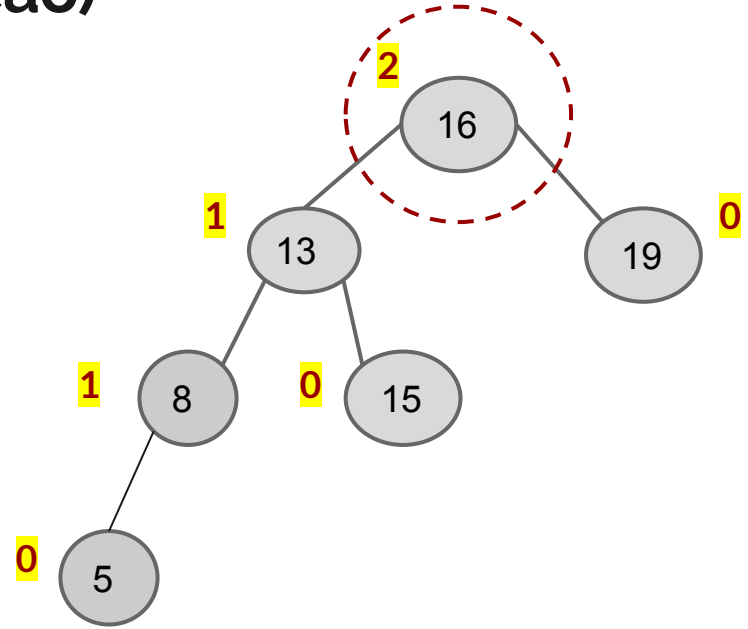


Árvores AVL (Inserção)

inserir(5)

Caso LL ⇒ R-rotation (root)

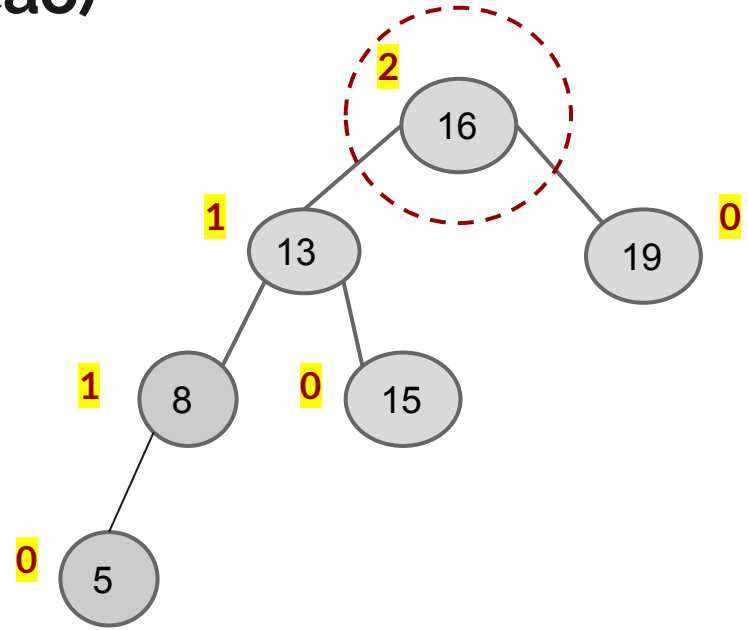
Como identificar o nó que violou o invariante?



Árvores AVL (Inserção)

inserir(5)

Caso LL ⇒ R-rotation (root)



Como identificar o nó que violou o invariante?

Recursão!

Árvores AVL (Inserção)

```
Insert (T element){
   Insert-recursive(root,element);
}

Insert-recursive(BSTNode node, T element){
  ...
}
```

```
Insert-recursive(BSTNode node, T element){
  if(node=NIL){
    node.data = element
    node.left = NIL
    node.right = NIL
  }else{
      if(element < node.data){
        Insert-recursive (node.left, element)
      }else if (element > node.data){
        Insert-recursive (node.right, element)
      }
      rebalance(node)
  }
}
```

Árvores AVL (Inserção)

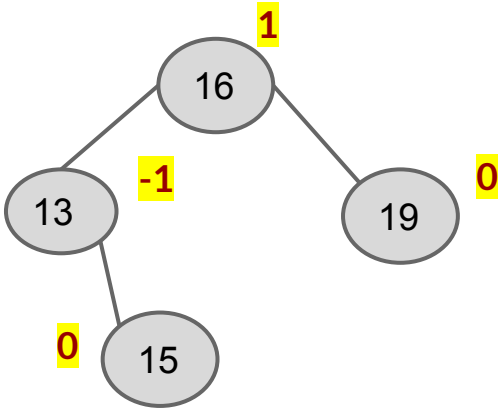
```
Insert (T element){
   Insert-recursive(root,element);
}

Insert-recursive(BSTNode node, T element){
  ...
}
```

```
Insert-recursive(BSTNode node, T element){
  if(node=NIL){
    node.data = element
    node.left = NIL
    node.right = NIL
  }else{
      if(element < node.data){
        Insert-recursive (node.left, element)
      }else if (element > node.data){
        Insert-recursive (node.right, element)
      }
      rebalance(node)
  }
}
```

Simulação

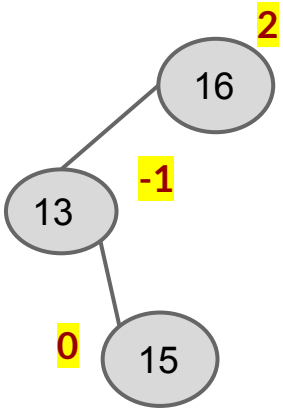
Árvores AVL (Remoção)

remover(19)



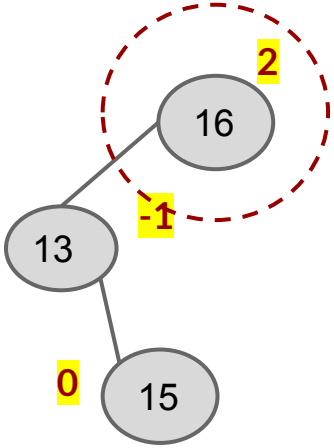
Árvores AVL (Remoção)

remover(19)



Árvores AVL (Remoção)

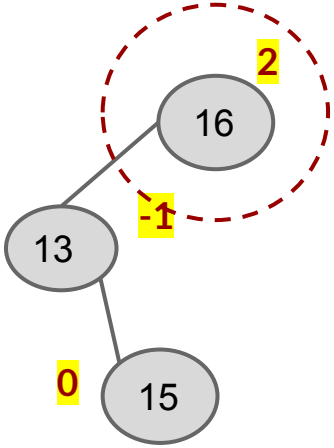
remover(19)



Árvores AVL (Remoção)

remover(19)

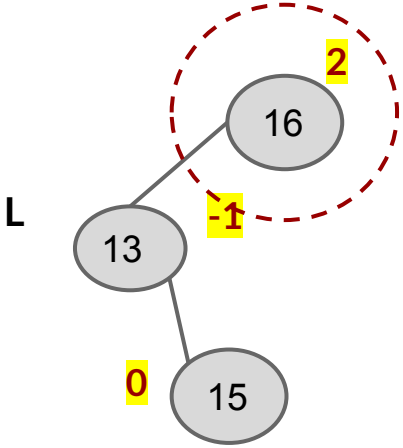
Caso LR ⇒ LR-rotation



Árvores AVL (Remoção)

remover(19)

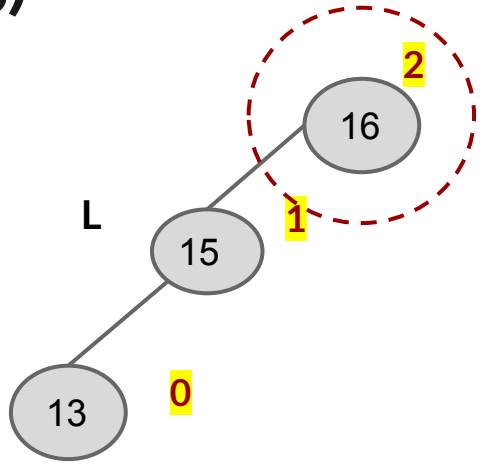
Caso LR ⇒ LR-rotation



Árvores AVL (Remoção)

remover(19)

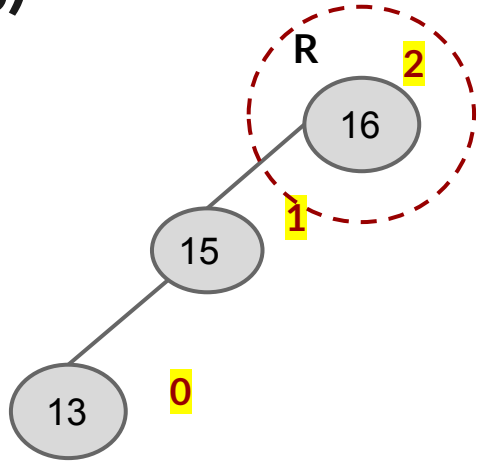
Caso LR ⇒ LR-rotation



Árvores AVL (Remoção)

remover(19)

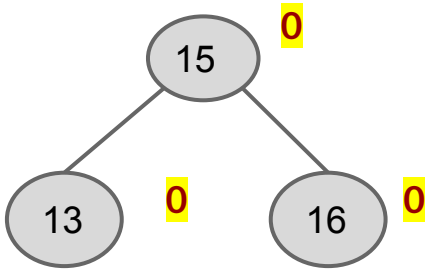
Caso LR ⇒ LR-rotation



Árvores AVL (Remoção)

remover(19)

Caso LR ⇒ LR-rotation



Árvores AVL (Remoção)

```
remove (T element){
   BSTNode<T> found = search(element);
   remove-recursive(found);
}

Remove-recursive(BSTNode node){
  ...
}
```

```
Remove-recursive(BSTNode node) {
  if(node != NIL){
    if(node is leaf){
      node = NIL
      rebalanceUp(node)
    }else if (node has one child){
      if node != root
        if(node is left child){
            if(node.left != NIL)
                node.left is left child of node.parent
            else
                node.right is left child of node.parent
        else  //node is right child
            if(node.left != NIL)
                node.left is right child of parent
            else
                node.right is right child of parent
      else
        root = not NIL child
      rebalanceUp(node)
    }else{
        BSTNode sucessor = sucessor(node);
        node.data = sucessor.data
        Remove-recursive (sucessor);
```

Árvores AVL (Remoção)

```
remove (T element){
   BSTNode<T> found = search(element);
   remove-recursive(found);
}

Remove-recursive(BSTNode node){
  ...
}
```

```
Remove-recursive(BSTNode node) {
  if(node != NIL){
    if(node is leaf){
      node = NIL
      rebalanceUp(node)
```

```
rebalanceUp(BSTNode<T> node) {
  BSTNode parent = node.parent
  while (parent != null) {
    rebalance(parent);
    parent = parent.parent;
  }
}
```

```
            node.left is right child of parent
          else
            node.right is right child of parent
      else
        root = not NIL child
      rebalanceUp(node)
    }else{
        BSTNode sucessor = sucessor(node);
        node.data = sucessor.data
        Remove-recursive (sucessor);
```

Árvores AVL (Rebalance)

```
void rebalance(BSTNode<T> node){
  balance = calculateBalance(node)
  se |balance| > 1
    determina o caso de balanceamento
    aplica a rotacao adequada (LL,RR,LR,RL)
}
```

Árvores AVL (Rebalance)

```
int calculateBalance(BSTNode<T> node) {
  if node != NIL
    return height (node.left) – height (node.right)
}
```

```
void rebalance(BSTNode<T> node){
  balance = calculateBalance(node)
  se |balance| > 1
    determina o caso de balanceamento
    aplica a rotacao adequada (LL,RR,LR,RL)
}
```

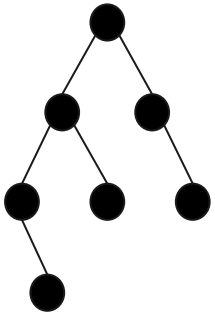
Árvores AVL (Rotações)

```
Pivot = Root.OS
Root.OS = Pivot.RS
Pivot.RS = Root
Root = Pivot
```

Onde implementar as rotações?

Árvores AVL (Rotações)

```
Pivot = Root.OS
Root.OS = Pivot.RS
Pivot.RS = Root
Root = Pivot
```



Onde implementar as rotações?

Componente externo

bt.Util.leftRotation(node)
bt.Util.rightRotation(node)

Árvores AVL (Rotações)

```
Pivot = Root.OS
Root.OS = Pivot.RS
Pivot.RS = Root
Root = Pivot
```

Ter atenção quando a rotação acontecer na raiz da árvore!

```
package adt.bt;

public class Util {
 public static <T extends Comparable<T>> BSTNode<T>
          leftRotation(BSTNode<T> node) {
   ...
 }
 public static <T extends Comparable<T>> BSTNode<T>
          rightRotation(BSTNode<T> node) {
   ...
 }
}
```

Árvores AVL (Aplicações)

Como ordenar um conjunto de dados (array) usando uma AVL?

Como calcular estatística de ordem usando uma AVL?

Busca binária pode ser feita com AVL?

Como inserir dados de um array em uma AVL evitando rotações?

Pode-se usar uma AVL nos problemas que se usa BST? Qual o impacto?

Árvore AVL
Implementação

Árvore AVL (Implementação)

Qual a relação entre AVL e BST?

O nó da AVL tem alguma diferença para o nó da BST?

É possível herdar interface e implementação?

Árvore AVL (Implementação)

```
public interface AVLTree<T extends Comparable<T>> extends BST<T>{
}
```

```
public interface AVLTreeImpl<T extends Comparable<T>>
    extends BSTImpl<T>{



}
```

Precisa sobrescrever algum método?
Precisa ter algum método específico?

Árvore AVL (Implementação)

```
public interface AVLTree<T extends Comparable<T>> extends BST<T>{
}
```

```
public interface AVLTreeImpl<T extends Comparable<T>>
    extends BSTImpl<T>{
    public void insert(T elem)
    public void remove(T elem)
    private int calculateBalance(BSTNode<T> node)
    private void rebalance(BSTNode<T> node)
    private void rebalanceUp(BSTNode<T> node)
}
```

Árvore AVL (Implementação das rotações)

```
package adt.bt;

public class Util {
 public static <T extends Comparable<T>>  BSTNode<T>
          leftRotation(BSTNode<T> node) {
   throw new UnsupportedOperationException("Not implemented yet!");
 }
 public static <T extends Comparable<T>>  BSTNode<T>
          rightRotation(BSTNode<T> node) {
   throw new UnsupportedOperationException("Not implemented yet!");
 }
}
```